

Faculty of Engineering
Department of Computer Engineering
University of Jaffna

EC9590: Network Application Design

Lab 05 & 06 — Task 02, Part 02

Session Management with Cookies

Student Information Management System (SIMS)

Name : Dayarathna D.D.R.N.
Date : 20 March 2026
Lecturer : Dr. J. Jananie
Department : Computer Engineering
Server : Apache Tomcat 10.1.52
Language : Java 21 (Temurin), Jakarta EE 6

March 20, 2026

Contents

1 Introduction

This report documents the implementation of session management with cookies in the **Student Information Management System (SIMS)**, a web-based application built using Jakarta Servlets, JSP, JDBC, and Apache Tomcat 10.1.

HTTP is a *stateless* protocol — each request is entirely independent and the server retains no memory of previous interactions. Session management bridges this gap by assigning each authenticated user a unique **Session ID**, stored in a browser cookie and verified on every subsequent request.

The implementation covers six explicitly defined stages:

1. **Login** — User submits credentials
2. **Creation** — Server creates a session record
3. **Transmission** — Server sends **Set-Cookie** header
4. **Persistence** — Browser attaches cookie to every request
5. **Validation** — Server verifies session on every page load
6. **Termination** — Server destroys session on logout

2 System Architecture

The project follows the **Model-View-Controller (MVC)** architectural pattern. The directory structure is as follows:

Listing 1: Project Structure

```
src/main/java/com/ec9590/sims/  
    model/  
        Student.java          -- Student entity  
        User.java             -- Login user entity  
    dao/  
        DBConnection.java     -- JDBC connection utility  
        StudentDAO.java       -- Student CRUD operations  
        UserDAO.java          -- Credential validation  
    servlet/  
        LoginServlet.java     -- Login (Creation +  
Transmission)  
        LogoutServlet.java    -- Logout (Termination)  
        ViewStudentServlet.java  
        AddStudentServlet.java  
        EditStudentServlet.java  
        UpdateStudentServlet.java  
        DeleteStudentServlet.java  
        SearchStudentServlet.java  
    filter/  
        SessionFilter.java     -- Validation on every request
```

3 Database Design

Two tables are used in the `sims_db` MySQL database.

3.1 Users Table

Listing 2: users table DDL

```
CREATE TABLE users (
  id          INT AUTO_INCREMENT PRIMARY KEY,
  username    VARCHAR(50)  UNIQUE NOT NULL,
  password    VARCHAR(255) NOT NULL
);

-- Default admin account
INSERT INTO users (username, password) VALUES ('admin', 'admin123');
```

3.2 Students Table

Listing 3: students table DDL

```
CREATE TABLE students (
  id          INT AUTO_INCREMENT PRIMARY KEY,
  reg_no      VARCHAR(20)  UNIQUE NOT NULL,
  name        VARCHAR(100) NOT NULL,
  field       VARCHAR(60)  NOT NULL,
  dob         DATE         NOT NULL,
  contact     VARCHAR(15),
  email       VARCHAR(100)
);
```

4 Session Management Implementation

4.1 Stage 1 — Login

Description: The user navigates to `/login.jsp` and submits their username and password via an HTML form using HTTP POST.

```
1 <form action="login" method="post">
2   <label>Username:</label>
3   <input type="text" name="username" required>
4
5   <label>Password:</label>
6   <input type="password" name="password" required>
7
8   <button type="submit">Login</button>
9 </form>
```

Listing 4: login.jsp – HTML form

`LoginServlet.doPost()` receives the credentials and validates them against the database using `UserDAO.validateUser()`, which uses a `PreparedStatement` to prevent SQL injection:

```
1 public User validateUser(String username, String password)
2     throws SQLException {
3     String sql = "SELECT * FROM users "
4         + "WHERE username = ? AND password = ?";
5     try (Connection con = DBConnection.getConnection();
6         PreparedStatement ps = con.prepareStatement(sql)) {
7
8         ps.setString(1, username);
9         ps.setString(2, password);
10
11         try (ResultSet rs = ps.executeQuery()) {
12             if (rs.next()) {
13                 return new User(rs.getInt("id"),
14                     rs.getString("username"),
15                     rs.getString("password"));
16             }
17         }
18     }
19     return null; // invalid credentials
20 }
```

Listing 5: UserDao.java – validateUser()

Screenshot: *[Insert screenshot of login page]*

4.2 Stage 2 — Creation

Description: Upon successful credential verification, the server creates a new server-side session record and maps the Session ID to the authenticated user.

```
1 if (user != null) {
2     // Invalidate old session to prevent session fixation attacks
3     HttpSession oldSession = req.getSession(false);
4     if (oldSession != null) oldSession.invalidate();
5
6     // Create new session -- Tomcat generates a unique Session ID
7     HttpSession session = req.getSession(true);
8
9     // Map Session ID to the authenticated user
10    session.setAttribute("loggedInUser", user.getUsername());
11
12    // Set session timeout: 30 minutes of inactivity
13    session.setMaxInactiveInterval(30 * 60);
14 }
```

Listing 6: LoginServlet.java – Session Creation

Internally, Tomcat maintains the following mapping:

Listing 7: Tomcat internal session store

```
SessionID_ABC123  -->  { loggedInUser: "admin",
                        createdAt: 2026-03-20T10:00:00,
                        lastAccessed: 2026-03-20T10:00:00 }
```

Screenshot: *[Insert screenshot of DevTools → Application → Cookies, showing JSESSIONID after login]*

4.3 Stage 3 — Transmission

Description: The server sends the Session ID to the browser via the Set-Cookie HTTP response header with HttpOnly and Secure flags.

```
1 // Create cookie carrying the Session ID
2 Cookie sessionCookie = new Cookie("sessionid", session.getId());
3 sessionCookie.setHttpOnly(true); // blocks JavaScript access
4 sessionCookie.setSecure(false); // set true in production (HTTPS)
5 sessionCookie.setPath(req.getContextPath());
6 sessionCookie.setMaxAge(30 * 60); // 30 minute lifetime
7
8 // Add Set-Cookie header to HTTP response
9 res.addCookie(sessionCookie);
10 res.sendRedirect(req.getContextPath() + "/viewStudents");
```

Listing 8: LoginServlet.java – Cookie Transmission

The resulting HTTP response header is:

Listing 9: HTTP Response Header

```
HTTP/1.1 302 Found
Set-Cookie: sessionid=ABC123XYZ; HttpOnly; Path=/SIMS; Max-Age=1800
Set-Cookie: JSESSIONID=ABC123XYZ; HttpOnly; Path=/SIMS
Location: http://localhost:8080/SIMS/viewStudents
```

Note on Secure flag: The Secure flag is set to `false` during development because the server runs on plain HTTP. In a production environment with HTTPS/TLS, this must be `true` to prevent the cookie from being transmitted over unencrypted connections.

Screenshot: *[Insert screenshot of DevTools → Network → login POST → Response Headers, showing Set-Cookie]*

4.4 Stage 4 — Persistence

Description: The browser stores the cookie and automatically attaches it to every subsequent HTTP request to the server.

After login, every request includes the cookie in its headers:

Listing 10: HTTP Request Header – Cookie sent automatically

```
GET /SIMS/viewStudents HTTP/1.1
Host: localhost:8080
Cookie: sessionid=ABC123XYZ; JSESSIONID=ABC123XYZ
```

The session attribute is available on every protected JSP page:

```
1 <!-- Session attribute accessible on every protected page -->
2 Welcome, <strong><%= session.getAttribute("loggedInUser") %></strong>
3 <a href="<%= request.getContextPath() %>/logout">Logout</a>
```

Listing 11: viewStudents.jsp – Session persistence demonstrated

Screenshot: *[Insert screenshot of View Students page showing “Welcome, admin” and DevTools → Network → Request Headers showing Cookie being sent]*

4.5 Stage 5 — Validation

Description: For every request to a protected resource, the server checks whether a valid session exists before allowing access.

This is implemented as a Jakarta Servlet Filter, which intercepts every request *before* it reaches any Servlet:

```
1 @WebFilter(urlPatterns = {
2     "/viewStudents", "/addStudent", "/editStudent",
3     "/updateStudent", "/deleteStudent", "/searchStudents",
4     "/addStudent.jsp", "/updateStudent.jsp",
5     "/viewStudents.jsp", "/index.jsp"
6 })
7 public class SessionFilter implements Filter {
8
9     @Override
10    public void doFilter(ServletRequest request,
11                        ServletResponse response,
12                        FilterChain chain)
13        throws IOException, ServletException {
14
15        HttpServletRequest req = (HttpServletRequest) request;
16        HttpServletResponse res = (HttpServletResponse) response;
17
18        // Retrieve existing session -- do NOT create a new one
19        HttpSession session = req.getSession(false);
20
21        boolean isLoggedIn = (session != null)
22                            && (session.getAttribute("loggedInUser")
23                                != null);
24
25        if (isLoggedIn) {
26            // Valid session -- allow request to proceed
27            chain.doFilter(request, response);
28        } else {
29            // No valid session -- block and redirect to login
30            res.sendRedirect(
31                req.getContextPath() + "/login.jsp");
32        }
33    }
34 }
```

Listing 12: SessionFilter.java – Validation on every request

Screenshot: *[Insert screenshot showing that accessing /viewStudents directly without login redirects to login.jsp]*

4.6 Stage 6 — Termination

Description: When the user clicks Logout, the server destroys the session data and instructs the browser to delete the cookie.

```
1 @WebServlet("/logout")
2 public class LogoutServlet extends HttpServlet {
3
4     @Override
5     protected void doGet(HttpServletRequest req,
```

```
6         HttpServletResponse res)
7         throws ServletException, IOException {
8
9         // Step 1: Destroy the server-side session record
10        HttpSession session = req.getSession(false);
11        if (session != null) {
12            session.invalidate(); // removes SessionID -> user
13        }
14
15        // Step 2: Delete sessionid cookie from browser
16        Cookie killCookie = new Cookie("sessionid", "");
17        killCookie.setMaxAge(0); // MaxAge=0 deletes the cookie
18        killCookie.setPath(req.getContextPath());
19        res.addCookie(killCookie);
20
21        // Step 3: Delete JSESSIONID cookie from browser
22        Cookie killJSession = new Cookie("JSESSIONID", "");
23        killJSession.setMaxAge(0);
24        killJSession.setPath(req.getContextPath());
25        res.addCookie(killJSession);
26
27        // Step 4: Redirect to login page
28        res.sendRedirect(req.getContextPath() + "/login.jsp");
29    }
30 }
```

Listing 13: LogoutServlet.java – Session Termination

The HTTP response after logout:

Listing 14: HTTP Response Header – Cookie deletion

```
HTTP/1.1 302 Found
Set-Cookie: sessionid=; Max-Age=0; Path=/SIMS
Set-Cookie: JSESSIONID=; Max-Age=0; Path=/SIMS
Location: http://localhost:8080/SIMS/login.jsp
```

Screenshot: *[Insert screenshot of DevTools → Network → logout → Response Headers showing Max-Age=0 cookies]*

5 Session Lifecycle Diagram

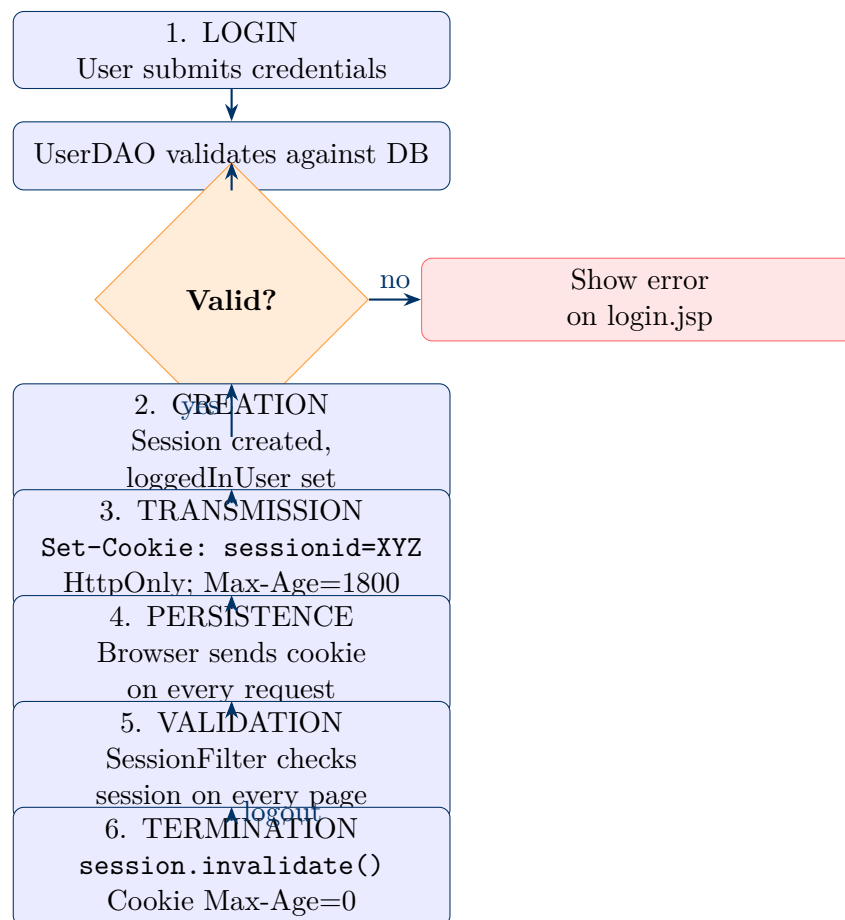


Figure 1: Complete Session Management Lifecycle in SIMS

6 Security Considerations

Security Concern	Implementation
SQL Injection	PreparedStatement used in all database queries
Session Fixation	Old session invalidated before creating a new one on login
XSS Cookie Theft	HttpOnly flag prevents JavaScript from reading the cookie
HTTPS Enforcement	Secure flag to be enabled in production with TLS
Session Timeout	30-minute inactivity timeout configured via <code>setMaxInactiveInterval()</code>
Unauthorised Access	SessionFilter blocks all unauthenticated requests to protected URLs

Table 1: Security Measures Implemented

7 Summary Table

Stage	Key File	Key Code
Login	LoginServlet.java	req.getParameter("username")
Creation	LoginServlet.java	req.getSession(true) + setAttribute()
Transmission	LoginServlet.java	new Cookie("sessionid", session.getId())
Persistence	viewStudents.jsp	session.getAttribute("loggedInUser")
Validation	SessionFilter.java	req.getSession(false) + null check
Termination	LogoutServlet.java	session.invalidate() + cookie.setMaxAge(0)

Table 2: Session Management Stage Summary

8 Conclusion

The SIMS application successfully implements all six stages of cookie-based session management as specified in the lab requirements. The `SessionFilter` provides centralised validation on every protected request, the `LoginServlet` handles secure session creation and cookie transmission, and the `LogoutServlet` ensures complete session termination on both the server and client sides.

The implementation demonstrates a foundational understanding of how stateful web sessions are managed over the inherently stateless HTTP protocol, and reflects the same mechanisms used internally by production-grade frameworks such as Spring Security.